

## UNIT VI: Web Services

---

### Q1. Discuss the role of SOAP web service in details.

**SOAP** stands for **Simple Object Access Protocol**. It is a **protocol** (a strict set of rules) for exchanging structured information between applications over a network. It is a core part of the traditional web services stack.

The primary role of SOAP is to provide a standard, language-independent way for applications to communicate, even if they are built on different platforms (e.g., a Java application on Linux communicating with a .NET application on Windows).

#### Key Roles and Features:

1. **Protocol-Based:** It is a formal protocol with strict rules. This ensures high reliability and consistency.
2. **XML-Based:** All SOAP messages are formatted using **XML**. This makes them human-readable and platform-independent.
3. **Transport Independence:** While most commonly used over HTTP, SOAP can technically run over other protocols like SMTP (email) or TCP.
4. **Built-in Error Handling:** The SOAP format includes a specific element, <Fault>, which is used to report errors in a standardized way.
5. **Support for Security and Transactions:** SOAP is highly extensible. It has standards like **WS-Security** that can be built on top of it to handle complex security requirements and transactions (e.g., for banking).

**SOAP Message Structure:** A SOAP message is an XML document with the following parts:

- **Envelope:** The mandatory root element that identifies the XML document as a SOAP message.
- **Header:** An optional element that contains metadata for the message. This is where extra features like security credentials, session management, or transaction details are placed.
- **Body:** A mandatory element that contains the actual application message (the data) being sent.
- **Fault:** An optional element used to provide information about errors that occurred while processing the message (only used if there is an error).

**Role:** SOAP is typically used in enterprise-level applications where reliability, security, and formal contracts (defined by WSDL) are more important than performance.

---

### Q2. What are Web services? Explain different types of Web Services.

A web service is a software system that allows two different applications (machines) to communicate with each other over a network, typically the internet. It is a method of **interoperability**, meaning a program written in one language (like Java) can call a service written in another language (like C# or PHP) without any problems.

Web services use standard web protocols (like HTTP) and data formats (like XML or JSON) to exchange information.

**Different Types of Web Services:** There are two main types (or architectural styles) of web services:

1. **SOAP (Simple Object Access Protocol):** This is a formal **protocol** with a strict set of rules. It relies exclusively on **XML** for its message format. It requires a formal "contract" file, called a **WSDL** (Web Services Description Language), to describe the service. It is generally considered "heavyweight" due to the XML overhead but provides high reliability and features like built-in security (WS-Security).  
*Use Case:* Enterprise applications, banking, or any system requiring high security and transactional integrity.
2. **REST (Representational State Transfer):** This is an **architectural style**, not a strict protocol. It is a set of guidelines. It is much simpler and more lightweight than SOAP. It uses standard HTTP methods for operations:
  - **GET:** To retrieve data.
  - **POST:** To create new data.
  - **PUT:** To update existing data.
  - **DELETE:** To delete data.

It is flexible in its data format, but it most commonly uses **JSON** (JavaScript Object Notation), which is lightweight and easy to parse. It can also use XML.

- *Use Case:* The vast majority of modern public APIs, mobile apps, and web applications.

### Q3. Differentiate between SOAP and REST.

| Basis for Comparison | SOAP (Simple Object Access Protocol)                            | REST (Representational State Transfer)                                                   |
|----------------------|-----------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Type                 | It is a <b>Protocol</b> with strict rules.                      | It is an <b>Architectural Style</b> with flexible guidelines.                            |
| Data Format          | Strictly <b>XML</b> only.                                       | Flexible. Can use <b>JSON</b> , XML, plain text, etc. (JSON is most common).             |
| Transport            | Can use any transport protocol (HTTP, SMTP, TCP).               | Almost always relies on <b>HTTP/HTTPS</b> .                                              |
| Contract             | Requires a formal contract file (WSDL) to describe the service. | No formal contract required. Can use standards like OpenAPI (Swagger) for documentation. |
| Performance          | "Heavyweight." More XML overhead, which can be slower.          | "Lightweight." Less overhead (especially with JSON), which is generally faster.          |
| Security             | Has its own security standards (e.g., WS-Security) built-in.    | Relies on the transport layer for security (e.g., HTTPS).                                |
| State                | Can be designed to be stateful.                                 | Intended to be <b>stateless</b> . Every request must contain all info needed.            |

|                 |                                                                     |                                                                     |
|-----------------|---------------------------------------------------------------------|---------------------------------------------------------------------|
| <b>Use Case</b> | Enterprise systems, banking, payments (high security, reliability). | Public APIs, mobile apps, web apps (high performance, flexibility). |
|-----------------|---------------------------------------------------------------------|---------------------------------------------------------------------|

#### Q4. What is semantic web service?

A **Semantic Web Service** (or "Semantic Web") is an advanced type of web service that adds **machine-readable meaning** (semantics) to its description.

A normal web service is described by a WSDL file, which tells a human developer *how* to call the service. However, a computer program cannot *understand* what the service actually *does*.

A semantic web service solves this by using additional standards (like **OWL-S** or **WSDL-S**) to link the service's description to a formal **ontology**. An ontology is a dictionary that defines the "meaning" of terms and their relationships.

**The Goal:** The goal is to make web services "smarter" so that software "agents" can automatically:

1. **Discover:** Find a service that matches a specific need (e.g., "find me a service that can book a flight from Pune to Delhi").
2. **Invoke:** Understand how to call the service (what data to send) without a human programmer.
3. **Compose:** Automatically combine multiple small services to achieve a larger, more complex task.

#### Q5. What is WSDL? Highlight its significance.

**WSDL** stands for **Web Services Description Language**. It is an **XML-based language** used to create a "user manual" or "contract" for a web service (specifically a SOAP web service).

This WSDL file is a document that describes *everything* an application needs to know to be able to communicate with that web service.

Specifically, it defines:

- **What the service does:** The operations (functions) it provides (e.g., `getWeather`, `checkStockPrice`).
- **What data it expects:** The data types and message formats for input and output.
- **How to communicate:** The protocol to use (e.g., SOAP).
- **Where to find it:** The endpoint (the URL) where the service is located.

#### Significance (Importance) of WSDL:

1. **Formal Contract:** It acts as a binding contract between the service provider and the service consumer. It guarantees what the service will provide.
2. **Interoperability:** Because WSDL is a standard, any platform (Java, .NET, Python, etc.) can read and understand it. This is what allows different technologies to communicate.
3. **Automation (Code Generation):** This is its most significant benefit. Developer tools can read a WSDL file and **automatically generate** all the "stub" or "proxy" code needed to call the service. This saves developers from having to write complex, low-level code to create and parse SOAP XML messages.
4. **Discovery:** WSDL files are what get published in a UDDI registry, allowing other businesses to find and understand the service.

## Q6. What is RPC? List out the component of web service architecture.

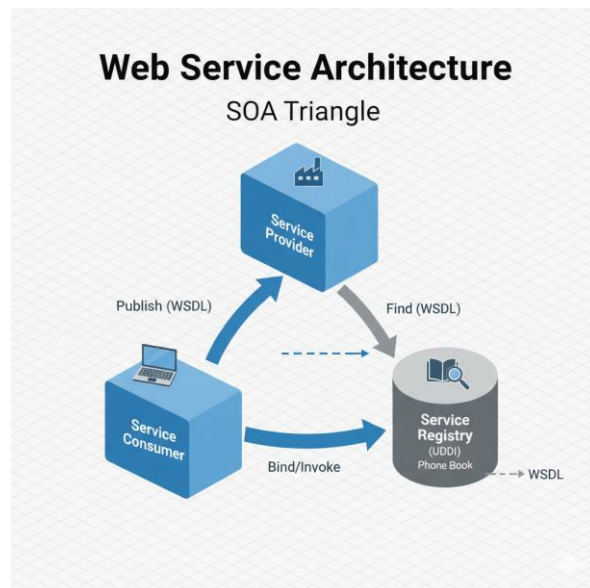
**RPC** stands for **Remote Procedure Call**. It is a programming model that allows a program to execute a function (a "procedure" or "subroutine") that is located on a *remote* computer on the network, just as if it were a normal, *local* function call.

The goal of RPC is to hide the complexity of the network communication. The developer simply calls a function (e.g., `getUserDetails(123)`) and the RPC system (called a "stub") handles:

1. Packaging the function name and parameters (this is called "marshalling").
2. Sending the request over the network.
3. Waiting for the remote server to execute the function.
4. Receiving the response.
5. Un-packaging the result ("unmarshalling") and returning it to the calling program.

### Components of Web Service Architecture:

The standard web service architecture (also called the "service-oriented architecture" or SOA triangle) has three main components:



1. **Service Provider (or Publisher):** This is the server or application that **creates** and **hosts** the web service. It is responsible for defining the service (using WSDL) and making it available on the network. It "publishes" the service description to a registry.
2. **Service Consumer (or Requester):** This is the client application that **needs** the service. It "finds" the service by searching the registry. It uses the WSDL to "bind" (connect) to the service and "invoke" (call) its functions.
3. **Service Registry (or Broker):** This is the "phone book" or "yellow pages" for web services (e.g., **UDDI**). It is a central directory where providers can advertise (publish) their services and consumers can discover them.

The three interactions are:

- **Publish:** The Provider publishes its service (WSDL) to the Registry.
- **Find:** The Consumer searches the Registry to find a service.
- **Bind/Invoke:** The Consumer uses the WSDL to connect to and use the Provider's service.

## Q7. What is semantic web service?

A **Semantic Web Service** (or "Semantic Web") is an advanced type of web service that adds **machine-readable meaning** (semantics) to its description.

A normal web service is described by a WSDL file, which tells a human developer *how* to call the service. However, a computer program cannot *understand* what the service actually *does*.

A semantic web service solves this by using additional standards (like **OWL-S** or **WSDL-S**) to link the service's description to a formal **ontology**. An ontology is a dictionary that defines the "meaning" of terms and their relationships.

**The Goal:** The goal is to make web services "smarter" so that software "agents" can automatically:

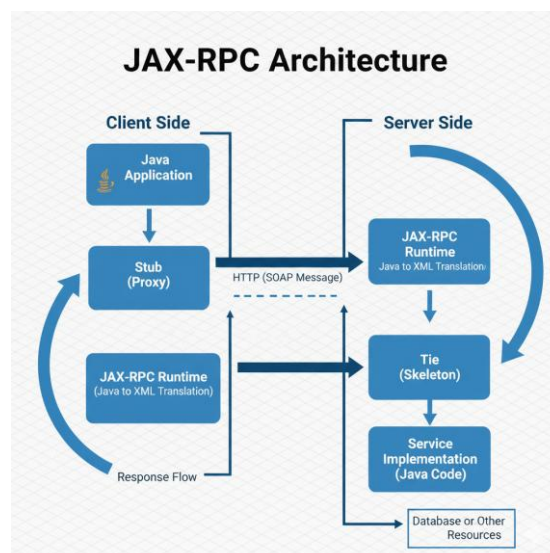
1. **Discover:** Find a service that matches a specific need (e.g., "find me a service that can book a flight from Pune to Delhi").
2. **Invoke:** Understand how to call the service (what data to send) without a human programmer.
3. **Compose:** Automatically combine multiple small services to achieve a larger, more complex task.

## Q8. What is JAX-RPC? Explain in detail with the help of diagram.

**JAX-RPC** stands for **Java API for XML-based RPC**.<sup>40</sup> It is an older Java API (a set of classes and libraries) used for building and consuming **SOAP web services**.<sup>41</sup>

**Purpose:** The main purpose of JAX-RPC is to hide the complexity of SOAP and XML from the Java developer. It allows a Java programmer to make a remote web service call as if it were a simple, local Java method call.

**How it Works (The Flow):** JAX-RPC uses "stubs" on the client side and "ties" on the server side to translate between Java and XML.



### 1. Client Side:

1. **Java App:** The client application (e.g., a desktop app) wants to call a web service.
2. **Stub:** The developer uses a **stub** (a local proxy object, usually generated from WSDL). The app calls a normal Java method on this stub (e.g., `stub.getWeather("Pune")`).
3. **JAX-RPC Runtime:** The stub passes the call to the JAX-RPC runtime system. This runtime converts the Java method call (`getWeather, "Pune"`) into a **SOAP (XML) message**.



4. **Network:** The runtime sends this XML message over HTTP to the server's URL.

## 2. Server Side:

1. **JAX-RPC Runtime:** The server's JAX-RPC runtime receives the incoming SOAP (XML) message.
2. **Tie (Skeleton):** The runtime passes the XML to a **tie** (or "skeleton") object.
3. **Java Implementation:** The tie "unmarshals" (converts) the XML back into a Java method call and invokes the *actual* service implementation (the business logic, e.g., a WeatherService.java class)
4. The Java method runs and returns a result (e.g., "25°C").
5. This response is passed back to the runtime, which converts it *back* into a SOAP (XML) response message and sends it to the client.

In summary, JAX-RPC acts as a two-way translator between the Java programming world and the XML-based SOAP world. (Note: JAX-RPC is now largely replaced by the more modern JAX-WS.)

---

## Q9. Explain the UDDI Registries? What are the uses of UDDI Registry?

**UDDI** stands for **Universal Description, Discovery, and Integration**.<sup>43</sup> It is an XML-based specification for a **Service Registry**. The simplest analogy for UDDI is a "**Yellow Pages**" or "**phone book**" for **web services**. It is a central directory where businesses (Service Providers) can **publish** information about the web services they offer, and other businesses (Service Consumers) can **discover** those services.

A UDDI registry stores information in three categories:

- **White Pages:** Basic contact information about the business (Name, Address, Phone Number).  
*Analogy: The alphabetical listings in a phone book.*
- **Yellow Pages:** Categorization information that groups services by industry, product, or location.  
*Analogy: The business categories in the Yellow Pages (e.g., "Plumbers," "Restaurants").*
- **Green Pages:** The technical information about the service. This includes the **WSDL** file, which describes *how* to call the service, and the service's endpoint (URL).

### Uses of UDDI Registry:

1. **Service Discovery:** This is the primary use. It allows a developer or application to search for a web service that meets a specific need (e.g., "find all services that provide credit card processing").
2. **Service Publishing (Advertising):** It gives companies a standard way to advertise their web services to the world, making them available for others to use (e.g., for B2B integration).
3. **Integration:** It simplifies the process of B2B (Business-to-Business) integration. A company can find a partner's WSDL in the registry, automatically generate the client code, and integrate their systems.
4. **Interoperability:** It promotes a standard, platform-independent way to find and describe services, which helps different systems work together.

---

## Q10. Describe the WSDL Structure and its life cycle in detail.

**WSDL Structure:** A WSDL (Web Services Description Language) file is an XML document. Its main purpose is to describe *what* a service does, *how* to call it, and *where* it is.

**The main elements of a WSDL file are:**

1. **<definitions>:** This is the **root element** of the entire WSDL document.
2. **<types>:** This section defines the **data types** used by the service. It usually uses XML Schema

(XSD) to define complex types (e.g., a "Student" object with name and roll\_no fields).

3. **<message>**: This defines the data (the "payload") being sent or received.<sup>49</sup> Each <message> element consists of one or more <part> elements, which are linked to the types defined in the <types> section.
4. **<portType>**: This is the most important abstract part. It defines a set of **operations** (methods or functions) that the service supports. For each operation, it specifies the input and output messages (e.g., getStudent operation takes a studentId\_Request message and returns a studentDetails\_Response message).
5. **<binding>**: This is the concrete part. It "binds" a portType to a specific **protocol** and **data format**. For example, it says, "use the getStudent operation over the SOAP 1.1 protocol with HTTP."
6. **<service>**: This defines the actual **endpoint** (the URL) where the service can be accessed. It groups one or more <port> elements, where each <port> links a specific binding to a physical address.

**WSDL Life Cycle:** The WSDL life cycle describes how a WSDL file is used from creation to execution.

1. **Define:** The service provider first **defines** the service. They either write the WSDL file by hand (contract-first) or write the server-side code (in Java or C#) and use a tool to **generate** the WSDL file from the code (code-first).
2. **Publish:** The provider **publishes** the WSDL file so consumers can find it. This can be as simple as placing it on a web server or as formal as publishing it in a **UDDI Registry**.
3. **Find (Discover):** The service consumer **finds** the WSDL file (e.g., by searching the UDDI registry or being given the URL).
4. **Generate:** The consumer uses a development tool (like wsimport in Java or "Add Service Reference" in .NET) to read the WSDL file.<sup>50</sup> The tool automatically **generates** client-side "stub" or "proxy" classes.
5. **Bind and Invoke:** The consumer's application code doesn't interact with XML. It simply calls the methods on the auto-generated stub class. This stub class handles all the work of "binding" to the service, creating the SOAP message, and "invoking" the remote service.

---

#### **Q11. Name any four web services technologies with explanation.**

1. **SOAP (Simple Object Access Protocol):** A formal, XML-based **protocol** (a set of rules) for exchanging structured data. It is the "message" format for many traditional web services. It is known for its reliability and security features (like WS-Security) and is often used in enterprise environments.
2. **WSDL (Web Services Description Language):** An XML-based "contract" or "user manual" for a web service. It describes *what* the service does (operations), *what data* it needs (messages and types), and *where* it is located (endpoint address). It is essential for enabling automation and interoperability.
3. **UDDI (Universal Description, Discovery, and Integration):** A "phone book" or **registry** for web services. It is an XML-based directory where businesses can publish their WSDL files (advertising their services) and other businesses can search to discover them.
4. **REST (Representational State Transfer):** An **architectural style** (not a protocol) that is a more lightweight and flexible alternative to SOAP. It uses standard HTTP methods (GET, POST, PUT, DELETE) and is not limited to XML (it most commonly uses **JSON**). REST has become the dominant style for public APIs, mobile apps, and modern web development.